

Universidade de São Paulo
Escola Politécnica da Universidade de São Paulo, Departamento de Engenharia de
Computação e Sistemas Digitais

EPUSP - PCS3848 - Sistemas Embarcados



Projeto Talos
Cofre Eletrônico com ESP32
Monografia

André Moreno - 13682785
Felipe Soria - 13864287
Giovani Sebastianutti - 13680171
Matheus Guidoni - 12683120

São Paulo,
2026

Resumo

Este trabalho apresenta o projeto, a implementação e os testes do Projeto Talos, um cofre eletrônico controlado por microcontrolador ESP32. O sistema permite o travamento e destravamento de uma fechadura eletromecânica por meio de autenticação por senha de seis dígitos, incorporando mecanismos de segurança como bloqueio progressivo após tentativas inválidas, desafio anti-marcas com dígitos aleatórios, modo stealth (silêncio sonoro), detecção automática de fechamento para trancamento e gestão de energia com modo de baixo consumo (deep sleep). O público-alvo do projeto é composto por usuários residenciais e pequenos estabelecimentos comerciais que necessitam de um sistema de proteção de baixo custo, simples de operar e com autonomia prolongada por bateria. A arquitetura do firmware foi modelada em duas camadas: Fluxo de Dados (abstração de hardware) e Unidade de Controle (máquina de estados). A persistência de senha e configurações utiliza memória não volátil (NVS) com proteção por CRC e esquema de cópia dupla. O documento detalha todos os requisitos funcionais e não funcionais, a especificação de hardware, os diagramas de estados e de blocos, os fluxos de operação e os critérios de aceitação, permitindo a replicação completa do projeto.

Palavras-chave: cofre eletrônico, ESP32, sistemas embarcados, máquina de estados, segurança, firmware e baixo consumo.

Sumário

Resumo	2
1 Introdução	5
1.1 Contextualização	5
1.2 Público-alvo	5
1.3 Objetivo	5
1.4 Escopo do firmware	5
1.5 Estrutura do documento	5
2 Premissas e Definições	6
3 Requisitos Funcionais	7
3.1 RF-01 – Debounce e evento singular	7
3.2 RF-02 – Timeout de inatividade	7
3.3 RF-03 – Bloqueio após tentativas limitadas	7
3.4 RF-04 – Bloqueio progressivo (backoff)	7
3.5 RF-05 – Liberação da tranca por senha correta	7
3.6 RF-06 – Troca de senha	7
3.7 RF-07 – Desafio anti-marcas	7
3.8 RF-08 – Feedback sonoro	7
3.9 RF-10 – Bateria fácil de trocar	8
3.10 RF-11 – Aviso de baixa bateria	8
3.11 RF-12 – Modo descanso (deep sleep)	8
3.12 RF-13 – Interação desativa modo descanso	8
3.13 RF-14 – Modo stealth (sem som)	8
3.14 RF-15 – Confirmar só com 6 dígitos	8
3.15 RF-16 – Persistência sem bateria	8
3.16 RF-17 – Detecção de fechamento e trancamento automático	8
3.17 RF-18 – Indicação de sucesso na autenticação	8
4 Requisitos Não Funcionais	9
4.1 RNF-01 – Confiabilidade extrema	9
4.2 RNF-02 – Segurança lógica	9
4.3 RNF-03 – Robustez da NVM	9
4.4 RNF-04 – Usabilidade	9
4.5 RNF-05 – Eficiência energética	9
4.6 RNF-06 – Hardware minimalista	9
4.7 RNF-07 – Carcaça resistente e eletrônica inacessível	9
5 Arquitetura do Sistema	10
5.1 Fluxo de Dados (Datapath)	10
5.2 Unidade de Controle	10
5.3 Interação entre camadas	10
6 Máquina de Estados do Firmware	11
6.1 S0 – Descanso (Deep Sleep)	11
6.2 S1 – Gerador de Entropia	11
6.3 S2 – Desafio Anti-marcas (Aguarda 2 dígitos)	11
6.4 S3 – Entrada de Senha (6 dígitos)	11
6.5 S4 – Validação	11
6.6 S5 – Acesso Concedido (Destruir)	11
6.7 S6 – Bloqueio (Lockout)	11
6.8 S7 – Trancamento Automático	11
6.9 S8 – Modo Troca de Senha	12
6.10 Tabela de transições	12
7 Fluxos de Operação	13
7.1 Fluxo de autenticação	13
7.2 Fluxo de troca de senha	13
7.3 Fluxo de modo stealth	13

8	Interface de Hardware	14
8.1	Microcontrolador	14
8.2	Entradas	14
8.3	Saídas	14
8.4	Circuito de medição de bateria	14
8.5	Energia e consumo	15
8.6	Esquemático de conexões	15
9	Persistência e Dados (NVM)	16
9.1	Dados armazenados	16
9.2	Esquema A/B para robustez	16
9.3	Armazenamento de senha como hash	16
10	Parâmetros Configuráveis	17
11	Critérios de Aceitação e Testes	18
12	Considerações Finais	19
	Referências	20

1 Introdução

1.1 Contextualização

Sistemas embarcados de segurança física, como cofres eletrônicos, demandam alta confiabilidade, baixo consumo energético e proteção contra diversas formas de ataque. Em ambientes domésticos e comerciais, cofres controlados eletronicamente oferecem vantagens sobre modelos puramente mecânicos, como a possibilidade de troca de senha, registro de eventos e modos de operação configuráveis. Entretanto, o projeto de tais sistemas requer atenção cuidadosa a aspectos como tratamento de entradas ruidosas (debounce), prevenção de ataques por força bruta, resistência a falhas de energia e proteção contra análise de padrões de uso nos botões.

1.2 Público-alvo

O Projeto Talos é direcionado principalmente a usuários residenciais, laboratórios acadêmicos e pequenos estabelecimentos comerciais que necessitam proteger objetos de valor ou documentos com um sistema eletrônico de baixo custo, operação intuitiva e manutenção simples. O projeto não tem como foco aplicações bancárias, militares ou corporativas de alta criticidade, nas quais seriam exigidos mecanismos adicionais de certificação, redundância e resistência física.

1.3 Objetivo

O Projeto Talos tem como objetivo implementar um cofre eletrônico controlado por um microcontrolador ESP32 que permita liberar e trancar uma fechadura eletromecânica por meio de autenticação por senha, com alta confiabilidade, baixo consumo energético, tratamento robusto de entradas e mecanismos anti-desgaste nos botões, atendendo ao contexto de uso de residências, ambientes acadêmicos e pequenos comércios. O sistema deve ser capaz de operar por pelo menos dois anos com uma única bateria em condições normais de uso.

1.4 Escopo do firmware

O firmware do Projeto Talos abrange as seguintes funcionalidades principais:

- Leitura de botões com debounce e validação de eventos, garantindo que cada pressionamento gere exatamente um evento.
- Máquina de estados com timeout de inatividade, modo descanso (deep sleep), bloqueio por tentativas e backoff progressivo.
- Fluxos de autenticação, troca de senha, modo stealth, limpeza de entrada e confirmação.
- Detecção de fechamento para trancamento automático via sensor (reed switch ou hall sensor).
- Gestão de energia e bateria com aviso de tensão baixa.
- Persistência de senha e configurações em memória não volátil (NVS do ESP32) com proteção CRC.

1.5 Estrutura do documento

Esta monografia está estruturada da seguinte forma: o Capítulo 2 apresenta as premissas e definições adotadas, o Capítulo 3 detalha os requisitos funcionais, o Capítulo 4 descreve os requisitos não funcionais, o Capítulo 5 apresenta a arquitetura do sistema, o Capítulo 6 detalha a máquina de estados do firmware, o Capítulo 7 descreve os fluxos de operação, o Capítulo 8 especifica a interface de hardware, o Capítulo 9 aborda a persistência de dados, o Capítulo 10 lista os parâmetros configuráveis, o Capítulo 11 apresenta os critérios de aceitação e testes, e o Capítulo 12 traz as considerações finais.

2 Premissas e Definições

Para garantir clareza e consistência ao longo de toda a especificação, apresentam-se a seguir as premissas adotadas e as definições dos termos utilizados neste documento.

Tabela 1: Glossário de termos e definições.

Termo	Definição
Senha	Sequência de exatamente 6 dígitos (0–9) utilizada para autenticar o usuário. Não é possível definir senhas com tamanho diferente de 6.
Entrada (buffer)	Sequência de dígitos digitada pelo usuário, armazenada temporariamente em memória volátil até a confirmação ou timeout.
Confirmar	Botão dedicado que submete a entrada atual para validação. Só é aceito quando o buffer contém exatamente 6 dígitos.
Limpar	Botão dedicado que apaga todo o conteúdo do buffer de entrada a qualquer momento antes da confirmação.
Timeout de inatividade	Tempo máximo (T_{inatividade}) sem interação do usuário antes que o sistema invalide a entrada parcial e retorne ao estado inicial.
Bloqueio (lockout)	Período (T_{lockout}) em que o cofre ignora todas as entradas após o usuário exceder o número máximo de tentativas inválidas (N_{tentativas}).
Desafio anti-marcas	Dois dígitos aleatórios gerados pelo sistema que o usuário deve digitar antes de inserir a senha, visando distribuir o desgaste entre os botões.
Modo stealth	Modo de operação em que todo o feedback sonoro é desativado, mantendo apenas indicações visuais (se existentes).
Modo descanso (sleep)	Estado de baixo consumo energético (deep sleep do ESP32) ativado após T_{descanso} segundos sem interação.
NVM / NVS	Memória não volátil (Non-Volatile Storage). No ESP32, utiliza-se a partição NVS da flash interna.

3 Requisitos Funcionais

Nesta seção são detalhados todos os requisitos funcionais do sistema, identificados com o prefixo RF. Cada requisito é acompanhado de sua descrição detalhada e critério de aceitação.

3.1 RF-01 – Debounce e evento singular

Cada pressionamento de botão deve gerar exatamente um evento, mesmo que haja bouncing elétrico no contato. O firmware implementa debounce por software com janela mínima de 50 ms entre eventos consecutivos de um mesmo botão. Após a detecção de borda de descida (botão pressionado), inicia-se um temporizador e, somente ao final deste período, se o botão ainda estiver pressionado, o evento é registrado. Critério: nenhuma repetição inadvertida de dígitos por um único toque, e pressionar e segurar não gera múltiplos dígitos.

3.2 RF-02 – Timeout de inatividade

O sistema invalida qualquer entrada parcial (buffer com menos de 6 dígitos) após decorrer **T_inatividade** segundos sem nenhuma interação do usuário. O buffer é zerado e o sistema retorna ao estado Aguardando (S1). Valor recomendado: **T_inatividade** = 15 s (configurável entre 10s e 30s).

3.3 RF-03 – Bloqueio após tentativas limitadas

Após **N_tentativas** tentativas consecutivas de senha incorreta, o sistema entra em estado de bloqueio (lockout) por **T_lockout** segundos. Durante este período, todas as entradas do teclado são ignoradas. Valor recomendado: **N_tentativas** = 5, **T_lockout_base** = 60 s.

3.4 RF-04 – Bloqueio progressivo (backoff)

Se um novo ciclo de lockout ocorrer dentro de uma janela de recorrência **W_recorrencia** após o último bloqueio, o tempo de lockout é multiplicado progressivamente segundo a fórmula:

$$T_{\text{lockout}} = \min(T_{\text{max}}, T_{\text{base}} \times 2^k)$$

onde k é o número de lockouts consecutivos dentro da janela. Valor recomendado: **W_recorrencia** = 15 min, **T_max** = 30 min.

3.5 RF-05 – Liberação da tranca por senha correta

Ao confirmar uma senha válida de 6 dígitos, o sistema aciona o atuador da tranca (solenóide, servo ou relé) para liberar a abertura por um tempo controlado **T_acionamento**. O sistema emite indicação sonora e/ou visual de sucesso. Valor recomendado: **T_acionamento** = 5 s.

3.6 RF-06 – Troca de senha

O sistema permite a troca de senha mediante um fluxo controlado. A troca só é permitida se o usuário se autenticou recentemente (dentro de uma janela **W_admin** = 60 s) ou mediante um modo físico protegido (jumper interno). O fluxo exige as seguintes etapas: (1) confirmação da senha atual, (2) digitação da nova senha, (3) repetição da nova senha para confirmação e (4) persistência em NVM com CRC. Senha padrão de fábrica: 000000.

3.7 RF-07 – Desafio anti-marcas

Antes de iniciar a digitação da senha, o sistema gera dois dígitos aleatórios D_1 e D_2 (0–9) e os exibe ao usuário (via LED numérico ou outro indicador). O usuário deve digitar D_1 e D_2 corretamente para prosseguir à entrada da senha. Se os dígitos estiverem incorretos, o desafio é reiniciado. Objetivo: distribuir uniformemente as marcas de uso entre todos os botões, impedindo que um terceiro deduza os dígitos utilizados na senha ao observar os botões com mais marcas de usos.

3.8 RF-08 – Feedback sonoro

O sistema emite sinais sonoros distintos por meio de um buzzer para os seguintes eventos: clique aceito (beep curto), erro de senha (dois beeps curtos), sucesso na autenticação (melodia ascendente), entrada em bloqueio (beep longo), bateria baixa (beep intermitente). Os padrões sonoros devem ser suficientemente distintos para permitir identificação sem ambiguidade.

3.9 RF-10 – Bateria fácil de trocar

O design mecânico deve prever um compartimento de bateria acessível pelo exterior (tampa ou compartimento dedicado), sem expor a eletrônica sensível. A troca de bateria deve poder ser realizada sem ferramentas especiais.

3.10 RF-11 – Aviso de baixa bateria

O sistema monitora a tensão da bateria por meio de um ADC com divisor resistivo. Quando a tensão cai abaixo de um limiar configurável V_{low} (recomendado: 3,3 V para bateria de lítio 3,7 V nominal), o sistema emite um alerta sonoro e/ou visual.

3.11 RF-12 – Modo descanso (deep sleep)

Após `T_descanso` segundos sem nenhuma interação, o ESP32 entra em modo deep sleep. O consumo em deep sleep do ESP32 é tipicamente inferior a $10\mu A$. O sistema acorda mediante interrupção GPIO gerada por qualquer botão ou pelo sensor de fechamento. Valor recomendado: `T_descanso` = 60 s.

3.12 RF-13 – Interação desativa modo descanso

Qualquer evento válido de botão acorda o sistema do modo descanso. Após o despertar, o sistema retorna ao estado de geração de entropia (S1).

3.13 RF-14 – Modo stealth (sem som)

O sistema permite alternar entre modo normal e modo stealth por meio de uma combinação de teclas específica (por exemplo, Limpar + Confirmar simultaneamente por 3 s). No modo stealth, todo feedback sonoro é suprimido. O flag é persistido em NVM.

3.14 RF-15 – Confirmar só com 6 dígitos

O botão Confirmar só dispara a validação quando o buffer contém exatamente 6 dígitos. Se pressionado com menos de 6 dígitos, o sistema emite um feedback de erro leve (beep curto duplo) e não realiza nenhuma validação.

3.15 RF-16 – Persistência sem bateria

A senha armazenada (como hash) e as configurações do sistema (flag stealth, contadores de lockout) permanecem na memória não volátil mesmo após remoção completa da bateria.

3.16 RF-17 – Detecção de fechamento e trancamento automático

Um sensor (reed switch, hall sensor ou fim de curso) detecta quando a porta do cofre é fechada. Ao detectar o fechamento, o sistema aciona automaticamente a tranca para o estado travado, caso ainda não esteja.

3.17 RF-18 – Indicação de sucesso na autenticação

Ao autenticar com sucesso, o sistema emite feedback (sonoro e/ou visual) e aciona a tranca para liberar a abertura.

4 Requisitos Não Funcionais

Os requisitos não funcionais definem as qualidades e restrições que o sistema deve atender, complementando os requisitos funcionais.

4.1 RNF-01 – Confiabilidade extrema

O sistema deve operar de forma determinística e previsível. Após qualquer reset (intencional ou não), o firmware deve retornar a um estado seguro (tranca fechada, buffer zerado, sem acesso concedido). Não é permitido nenhum comportamento indefinido ou não determinístico na máquina de estados.

4.2 RNF-02 – Segurança lógica

O sistema não deve expor informações que auxiliem um atacante. Para isso, não deve indicar quantos dígitos estão corretos, deve utilizar comparação de senha em tempo constante para evitar ataques de temporização (*timing attacks*), deve armazenar a senha como hash, e não em texto claro, na NVM, e não deve incluir a senha em logs.

4.3 RNF-03 – Robustez da NVM

As escritas em memória não volátil devem ser protegidas contra corrupção por meio de CRC32 e esquema de cópia dupla (A/B). Cada escrita atualiza a cópia inativa, verifica o CRC, e somente então promove a cópia como ativa. Em caso de corrupção de uma cópia, a outra é utilizada.

4.4 RNF-04 – Usabilidade

O sistema deve fornecer feedback claro e consistente para todas as ações: clique aceito, erro, sucesso, bloqueio, bateria baixa e estado stealth. Os botões Limpar e Confirmar devem sempre executar suas funções esperadas, sem ambiguidade.

4.5 RNF-05 – Eficiência energética

O firmware deve priorizar baixo consumo em repouso utilizando o modo deep sleep do ESP32 e mantendo o mínimo de periféricos ativos. O despertar é realizado por GPIO (botões e sensor de fechamento). A meta é que a bateria dure pelo menos 2 anos em condições de uso padrão (estimativa: 5 acessos por dia).

4.6 RNF-06 – Hardware minimalista

O projeto deve minimizar a quantidade de componentes externos, aproveitando ao máximo os recursos internos do ESP32 (GPIOs, ADC, NVS, RNG). Componentes externos limitam-se ao teclado, buzzer, atuador da tranca, sensor de fechamento, divisor resistivo para leitura de bateria e indicador opcional (LED).

4.7 RNF-07 – Carcaça resistente e eletrônica inacessível

O design mecânico deve garantir que a eletrônica não seja acessível sem violar a carcaça do cofre. O compartimento de bateria é a única abertura externa e não deve permitir acesso à placa controladora.

5 Arquitetura do Sistema

A arquitetura do Projeto Talos foi modelada segundo a divisão de sistemas digitais em Fluxo de Dados (*Datapath*) e Unidade de Controle (*Control Unit*). Essa separação permite modularidade, facilita testes unitários e torna o sistema mais compreensível.

5.1 Fluxo de Dados (Datapath)

O Fluxo de Dados atua como a camada de abstração de hardware e processamento. Ele encapsula todos os módulos que interagem diretamente com o hardware ou realizam processamento de dados, recebendo comandos de controle da Unidade de Controle e retornando flags de status. Os módulos que compõem o Fluxo de Dados são:

- **Módulo Teclado:** responsável pela leitura dos botões físicos (0–9, Confirmar, Limpar) com debounce por software. Recebe o comando `en_limpar_buffer` e os sinais das GPIOs. Produz as flags `flag_teccla_valida` e `id_teccla` (identificação do botão pressionado).
- **Buffer de Entrada:** armazena temporariamente a sequência de dígitos digitada pelo usuário. Recebe comandos `en_salva_digito` e `en_limpar_buffer`. Produz a flag `flag_buffer_cheio` quando o buffer atinge 6 dígitos, e fornece os dados do buffer (`buffer_dados`) para os módulos de comparação.
- **Gerador e Comparador de Entropia:** gera os dois dígitos aleatórios D_1 e D_2 para o desafio anti-marcas utilizando o gerador de números aleatórios do hardware do ESP32 (`esp_random()`). Recebe comandos `en_gerar_entropia` e `compara_anti_marcas`. Produz as flags `flag_entropia_ok` e `flag_entropia_erro`, além do sinal para o LED numérico.
- **Comparador e Memória:** responsável por comparar a senha digitada com a senha armazenada em NVM. A comparação é realizada em tempo constante para evitar *timing attacks*. Recebe o comando `compara_senhas` e os dados do buffer. Produz as flags `flag_senha_ok` e `flag_senha_erro`.
- **Contador de Tentativas e Backoff:** mantém o contador de tentativas falhas consecutivas e calcula o tempo de lockout progressivo. Recebe comandos `count_falhas`, `zera_falhas` e `en_calcula_backoff`. Produz `flag_max_falhas_atingido` e o valor `tempo_lockout_calculado`.
- **Timers:** gerencia os temporizadores do sistema (`T_inatividade`, `T_descanso`, `T_lockout`, `T_acionamento`). Recebe o comando `en_timer` e o valor de tempo. Produz as flags `flag_timeout_inatividade` e `flag_timeout_lockout`.
- **Atenuador (Buzzer):** controla o buzzer para feedback sonoro. Recebe comandos da Unidade de Controle e o flag do modo stealth. Quando stealth está ativo, suprime toda saída sonora.
- **Sensor:** monitora o sensor de fechamento da porta (reed switch ou hall sensor). Produz a flag `flag_porta_fechada`.
- **Indicador de Bateria:** lê a tensão da bateria via ADC com divisor resistivo e aciona o LED de bateria baixa quando a tensão cai abaixo do limiar `V_low`.

5.2 Unidade de Controle

A Unidade de Controle centraliza a Máquina de Estados Finitos (FSM) descrita no Capítulo 6. Ela recebe todas as flags de status do Fluxo de Dados e emite comandos de controle para orquestrar o funcionamento do sistema.

Entradas (flags de status): `flag_teccla_valida`, `id_teccla`, `flag_buffer_cheio`, `flag_entropia_ok`, `flag_entropia_erro`, `flag_senha_ok`, `flag_senha_erro`, `flag_max_falhas_atingido`, `flag_timeout_inatividade`, `flag_timeout_lockout`, `flag_porta_fechada`.

Saídas (comandos de controle): `zera_entrada_da_senha`, `en_salva_digito`, `en_gerar_entropia`, `compara_anti_marcas`, `compara_senhas`, `count_falhas`, `zera_falhas`, `en_calcula_backoff`, `en_timer`, `en_aciona_tranca`.

5.3 Interação entre camadas

A comunicação entre Fluxo de Dados e Unidade de Controle é realizada por meio de sinais internos unidirecionais. O Fluxo de Dados fornece flags de status (informações sobre o estado atual do hardware e processamento) para a Unidade de Controle, que por sua vez emite comandos de controle (enables e triggers) para os módulos do Fluxo de Dados. Essa separação garante que a lógica de decisão esteja completamente isolada da manipulação de hardware.

6 Máquina de Estados do Firmware

A máquina de estados finitos (FSM) do firmware governa todo o comportamento do cofre. Cada estado é descrito a seguir com suas condições de entrada, ações e transições.

6.1 S0 – Descanso (Deep Sleep)

Estado de consumo mínimo. O ESP32 está em modo deep sleep.

Ações na entrada: desabilitar todos os periféricos não essenciais, configurar GPIOs como fonte de wake-up e entrar em deep sleep.

Transição: GPIO wake-up (botão ou sensor) → S1.

6.2 S1 – Gerador de Entropia

O sistema gera dois dígitos aleatórios D_1 e D_2 (0–9) utilizando o RNG do hardware do ESP32. Os dígitos são exibidos ao usuário por meio do LED numérico.

Ações: `en_gerar_entropia` = 1 e exibir D_1 e D_2 .

Transições: conclusão da geração → S2, e timeout sem interação → S0.

6.3 S2 – Desafio Anti-marcas (Aguarda 2 dígitos)

Aguarda o usuário digitar os dois dígitos do desafio. Cada dígito digitado é comparado individualmente com o esperado.

Ações: `compara_anti_marcas` = 1 a cada tecla.

Transições: ambos corretos (`flag_entropia_ok`) → S3, erro (`flag_entropia_erro`) → reinicia S1 (pode contar como tentativa, configurável), e timeout → S0.

6.4 S3 – Entrada de Senha (6 dígitos)

O sistema coleta os 6 dígitos da senha no buffer. A cada tecla válida, `en_salva_digito` é ativado. O botão Limpar (`en_limpar_buffer`) apaga todo o buffer. O timer de inatividade é reiniciado a cada tecla.

Ações: `en_salva_digito` e `en_timer` (reset inatividade).

Transições: Confirmar pressionado com `flag_buffer_cheio` → S4, timeout → limpa buffer e vai para S0, e Limpar → permanece em S3 com buffer zerado.

6.5 S4 – Validação

O sistema compara a senha digitada com a armazenada em NVM usando comparação em tempo constante.

Ações: `compara_senhas` = 1.

Transições: `flag_senha_ok` → S5 (zera falhas), e `flag_senha_erro` → `count_falhas`. Se `flag_max_falhas atingido` a transição vai para S6. Caso contrário, vai para S1.

6.6 S5 – Acesso Concedido (Destruar)

A tranca é acionada para liberar a abertura. Um timer `T_acionamento` é iniciado. Feedback sonoro de sucesso é emitido (salvo em modo stealth).

Ações: `en_aciona_tranca` = 1 (abrir), `en_timer(T_acionamento)` e `zera_falhas`.

Transições: `flag_porta_fechada` (sensor detecta fechamento) → S7, e `T_acionamento` expirado sem fechamento → S7 (tranca novamente).

6.7 S6 – Bloqueio (Lockout)

O sistema entra em lockout. Todas as entradas de teclado são ignoradas. O buzzer emite sinal de bloqueio (se não stealth).

Ações: `en_calcula_backoff` = 1, `en_timer(tempo_lockout_calculado)` e sinalizar estado de bloqueio.

Transições: `flag_timeout_lockout` (timer expirado) → S1.

6.8 S7 – Trancamento Automático

Ao detectar o fechamento da porta ou após expirar `T_acionamento`, o sistema aciona a tranca para o estado travado.

Ações: `en_aciona_tranca` = 1 (fechar) e confirmar mecanicamente o travamento.

Transições: travamento confirmado $\rightarrow$ S0 (entra em descanso) ou S1 (aguarda nova interação).

6.9 S8 – Modo Troca de Senha

Fluxo controlado para alteração da senha. Só acessível dentro de `W_admin` após autenticação bem-sucedida ou via jumper interno.

Fluxo: solicitar senha atual $\rightarrow$ validar $\rightarrow$ solicitar nova senha $\rightarrow$ solicitar repetição $\rightarrow$ persistir em NVM.

Transições: sucesso $\rightarrow$ S1, e falha ou timeout $\rightarrow$ S1.

6.10 Tabela de transições

A Tabela 2 resume as transições principais da máquina de estados.

Tabela 2: Transições da máquina de estados.

Origem	Condição	Destino
S0 (Sleep)	GPIO wake-up (botão ou sensor)	S1
S1 (Gera entropia)	Entropia gerada	S2
S1	Timeout sem interação	S0
S2 (Desafio)	Dígitos corretos	S3
S2	Dígitos incorretos	S1 (reinicia)
S2	Timeout	S0
S3 (Entrada senha)	Confirmar + buffer cheio	S4
S3	Timeout	S0
S3	Limpar	S3 (buffer zerado)
S4 (Validação)	Senha correta	S5
S4	Senha incorreta ($< N$)	S1
S4	Senha incorreta ($\geq N$)	S6
S5 (Acesso)	Porta fechada / timeout	S7
S6 (Lockout)	Timer expirado	S1
S7 (Tranca)	Travamento confirmado	S0 / S1

7 Fluxos de Operação

7.1 Fluxo de autenticação

O fluxo de autenticação é o fluxo principal de uso do cofre. Ele descreve a sequência completa de operações desde o despertar até a liberação da tranca:

1. O usuário pressiona qualquer botão, acordando o sistema do modo deep sleep ($S0 \rightarrow S1$).
2. O sistema gera dois dígitos aleatórios D_1 e D_2 e os exibe ao usuário ($S1 \rightarrow S2$).
3. O usuário digita D_1 e D_2 . Se corretos, o sistema prossegue para a entrada de senha ($S2 \rightarrow S3$). Se incorretos, reinicia o desafio ($S2 \rightarrow S1$).
4. O usuário digita os 6 dígitos da senha. A qualquer momento pode pressionar Limpar para apagar o buffer.
5. O usuário pressiona Confirmar. Se o buffer contém exatamente 6 dígitos, o sistema inicia a validação ($S3 \rightarrow S4$).
6. Se a senha está correta: a tranca é liberada, feedback de sucesso é emitido, e o contador de falhas é zerado ($S4 \rightarrow S5$).
7. Se a senha está incorreta: o contador de falhas é incrementado. Se atingiu $N_tentativas$, entra em lockout ($S4 \rightarrow S6$). Caso contrário, retorna ao início ($S4 \rightarrow S1$).
8. Após a abertura, quando o sensor detecta o fechamento da porta ou $T_acionamento$ expira, o sistema tranca automaticamente ($S5 \rightarrow S7 \rightarrow S0$).

7.2 Fluxo de troca de senha

A troca de senha segue um protocolo controlado para evitar alterações não autorizadas:

1. O usuário deve ter se autenticado recentemente (dentro de $W_admin = 60$ s) ou ativar o modo físico protegido (jumper interno). O sistema entra no estado S8.
2. O sistema solicita a senha atual. O usuário digita 6 dígitos e pressiona Confirmar.
3. Se a senha atual estiver correta, o sistema solicita a nova senha (6 dígitos + Confirmar).
4. O sistema solicita a repetição da nova senha para confirmação.
5. Se ambas coincidirem, a nova senha é persistida em NVM. Calcula-se o hash da nova senha, atualiza-se a cópia inativa (esquema A/B), verifica-se o CRC e promove-se a cópia como ativa.
6. Um evento de alteração de senha é registrado nos logs (sem armazenar a senha). O sistema retorna a S1.

7.3 Fluxo de modo stealth

O modo stealth pode ser alternado a qualquer momento por meio de uma combinação de teclas específica (por exemplo, pressionar Limpar e Confirmar simultaneamente por 3 segundos). Ao alternar, o flag stealth é invertido e persistido em NVM. Quando ativo, todo feedback sonoro é suprimido em todos os estados. Indicações visuais (LEDs) permanecem operacionais. O usuário recebe uma confirmação visual da alteração (por exemplo, LED piscando duas vezes).

8 Interface de Hardware

Esta seção descreve todos os componentes de hardware necessários para replicação do projeto, suas conexões e especificações.

8.1 Microcontrolador

O microcontrolador utilizado é o ESP32 (Espressif Systems), escolhido pelas seguintes características:

- Processador dual-core Xtensa LX6 a 240 MHz.
- Múltiplas GPIOs com suporte a interrupção e wake-up.
- ADC de 12 bits para leitura de tensão da bateria.
- Gerador de números aleatórios por hardware (RNG).
- Partição NVS em flash interna para armazenamento não volátil.
- Modos de baixo consumo (light sleep e deep sleep com consumo inferior a $10\mu\text{A}$).
- Framework ESP-IDF com suporte robusto a timers, GPIOs e NVS.

8.2 Entradas

Tabela 3: Componentes de entrada.

Componente	Qtd.	Descrição	Conexão
Botões numéricos (0–9)	10	Botões tácteis (push-button) com pull-up interno. Lógica: LOW quando pressionado.	GPIOs INPUT_PULLUP
Botão Confirmar	1	Botão táctil dedicado para submeter a entrada.	GPIO INPUT_PULLUP
Botão Limpar	1	Botão táctil dedicado para apagar o buffer.	GPIO INPUT_PULLUP
Sensor de fechamento	1	Reed switch ou hall sensor para detectar porta fechada. Lógica: LOW quando fechado.	GPIO INPUT_PULLUP + wake-up

Todos os botões utilizam o resistor de pull-up interno do ESP32 (aproximadamente $45\text{k}\Omega$). Para o debounce por software, é implementada uma janela de 50 ms conforme descrito no RF-01. Os botões podem opcionalmente ser organizados em matriz (*keypad matrix*) para reduzir o número de GPIOs utilizados, embora a implementação de referência utilize uma GPIO por botão para simplicidade.

8.3 Saídas

Tabela 4: Componentes de saída.

Componente	Qtd.	Descrição	Conexão
Atuador da tranca	1	Solenoide, servo motor ou relé.	GPIO + driver MOSFET/relé
Buzzer	1	Buzzer ativo ou passivo para feedback sonoro. Desativável pelo modo stealth.	GPIO (PWM p/ passivo)
LED numérico	1	Display de 7 segmentos ou LEDs para exibir dígitos do desafio anti-marcas.	GPIOs ou shift register
LED de bateria	1	LED indicador de bateria baixa (opcional).	GPIO

8.4 Circuito de medição de bateria

A tensão da bateria é medida por meio de um divisor resistivo conectado a uma entrada ADC do ESP32. Como a tensão nominal da bateria de lítio (3,7 V, podendo chegar a 4,2 V carregada) pode exceder a faixa do ADC do ESP32 (0–3,3 V na atenuação padrão), o divisor resistivo é necessário.

Recomenda-se utilizar resistores de alto valor (por exemplo, $R_1 = 100\text{k}\Omega$ e $R_2 = 100\text{k}\Omega$) para minimizar o consumo estático. A tensão lida pelo ADC será:

$$V_{\text{adc}} = V_{\text{bat}} \times \frac{R_2}{R_1 + R_2}$$

Com $R_1 = R_2 = 100\text{k}\Omega$, tem-se $V_{\text{adc}} = V_{\text{bat}}/2$. O limiar de bateria baixa `V_low` deve ser configurado considerando esta relação.

8.5 Energia e consumo

O sistema é alimentado por uma bateria de lítio de 3,7 V (tipo 18650 ou LiPo). A estimativa de consumo é apresentada na Tabela 5.

Tabela 5: Estimativa de consumo por modo de operação.

Modo	Corrente estimada	Duração típica
Deep sleep	$< 10 \mu\text{A}$	$\sim 99,9\%$ do tempo
Ativo (aguardando entrada)	$\sim 30 \text{ mA}$	$\sim 30 \text{ s}$ por acesso
Acionamento da tranca	$\sim 200\text{--}500 \text{ mA}$	$\sim 1 \text{ s}$ por acesso

Considerando uma bateria de 3000 mAh, 5 acessos por dia (cada acesso consumindo aproximadamente 30 s ativos + 1 s de acionamento), o consumo diário é dominado pelo modo deep sleep. A estimativa conservadora indica uma autonomia superior a 2 anos.

8.6 Esquemático de conexões

Para replicar o projeto, conectar os componentes conforme a Tabela 6. As GPIOs específicas podem ser ajustadas conforme a placa ESP32 utilizada, respeitando as seguintes restrições: evitar GPIOs 6–11 (conectadas à flash SPI interna) e preferir GPIOs que suportam wake-up em deep sleep (GPIOs 0, 2, 4, 12–15, 25–27, 32–39).

Tabela 6: Mapeamento de sinais e GPIOs.

Sinal	GPIO	Direção	Observação
Botão 0	GPIO 32	Entrada	Pull-up interno, wake-up compatível
Botão 1	GPIO 33	Entrada	Pull-up interno
Botão 2	GPIO 25	Entrada	Pull-up interno
Botão 3	GPIO 26	Entrada	Pull-up interno
Botão 4	GPIO 27	Entrada	Pull-up interno
Botão 5	GPIO 14	Entrada	Pull-up interno
Botão 6	GPIO 12	Entrada	Pull-up interno, atenção ao boot
Botão 7	GPIO 13	Entrada	Pull-up interno
Botão 8	GPIO 15	Entrada	Pull-up interno, atenção ao boot
Botão 9	GPIO 4	Entrada	Pull-up interno
Confirmar	GPIO 2	Entrada	Pull-up interno
Limpar	GPIO 0	Entrada	Pull-up interno, wake-up
Sensor fechamento	GPIO 34	Entrada	Somente entrada (sem pull-up interno, usar externo)
Tranca (driver)	GPIO 16	Saída	Via MOSFET ou relé
Buzzer	GPIO 17	Saída	PWM para buzzer passivo
LED numérico (dados)	GPIO 18	Saída	Shift register ou direto
LED numérico (clock)	GPIO 19	Saída	Se usar shift register
ADC bateria	GPIO 36 (VP)	Entrada	ADC1, somente entrada

9 Persistência e Dados (NVM)

A persistência de dados é um aspecto crítico do projeto, pois a senha e as configurações devem sobreviver a remoções de bateria e resets inesperados.

9.1 Dados armazenados

Os seguintes dados são persistidos na partição NVS do ESP32:

Tabela 7: Estrutura de dados persistidos em NVM.

Campo	Tipo	Tamanho	Descrição
<code>version</code>	<code>uint8_t</code>	1 byte	Versão do formato de dados. Permite migrações futuras.
<code>crc32</code>	<code>uint32_t</code>	4 bytes	CRC32 dos campos de dados para detecção de corrupção.
<code>senha_hash</code>	<code>uint8_t[32]</code>	32 bytes	Hash SHA-256 da senha de 6 dígitos.
<code>flag_stealth</code>	<code>uint8_t</code>	1 byte	0 = modo normal, 1 = modo stealth.
<code>lockout_count</code>	<code>uint8_t</code>	1 byte	Contador de lockouts consecutivos na janela de recorrência.
<code>last_lockout_ts</code>	<code>uint32_t</code>	4 bytes	Timestamp (uptime-based) do último lockout.

9.2 Esquema A/B para robustez

Para proteger contra corrupção por interrupção de escrita (queda de bateria durante gravação), utiliza-se o esquema de cópia dupla:

1. Duas cópias (A e B) da estrutura de dados são mantidas em chaves NVS distintas (por exemplo, `config_a` e `config_b`).
2. Cada cópia inclui um campo `version` e um campo `crc32`.
3. Na inicialização, lê-se ambas as cópias, verifica-se o CRC de cada uma e utiliza-se a cópia com CRC válido e `version` mais recente.
4. Na escrita, incrementa-se a `version`, calcula-se o CRC32 dos dados, grava-se na cópia **inativa**, lê-se de volta e verifica-se o CRC. Somente então a cópia recém-gravada torna-se a ativa.
5. Se ambas as cópias estão corrompidas na inicialização, o sistema assume valores padrão (senha = 000000, stealth = desligado) e regrava ambas as cópias.

9.3 Armazenamento de senha como hash

A senha nunca é armazenada em texto claro. Utiliza-se o algoritmo SHA-256 para gerar o hash da senha de 6 dígitos. Na validação, o hash da entrada é comparado com o hash armazenado usando comparação em tempo constante (função que itera sobre todos os bytes independentemente de diferenças encontradas).

Embora o espaço de senhas de 6 dígitos seja relativamente pequeno ($10^6 = 1.000.000$ de combinações), o hash impede a leitura direta da senha caso a flash seja extraída fisicamente, e o bloqueio progressivo (RF-04) torna ataques de força bruta impraticáveis via interface normal.

10 Parâmetros Configuráveis

Os parâmetros a seguir podem ser definidos como constantes no código-fonte (`#define` ou `constexpr`) ou, opcionalmente, ajustados via modo administrativo. A Tabela 8 apresenta os valores recomendados e as faixas aceitáveis.

Tabela 8: Parâmetros configuráveis do sistema.

Parâmetro	Símbolo	Padrão	Faixa	Descrição
Timeout de inatividade	<code>T_inat</code>	15 s	10–30 s	Tempo máximo sem interação para invalidar entrada parcial.
Tempo para descanso	<code>T_desc</code>	60 s	30–120 s	Tempo sem interação para entrar em deep sleep.
Máx. tentativas	<code>N_tent</code>	5	3–10	Tentativas falhas antes do bloqueio.
Lockout base	<code>T_lock</code>	60 s	30 s–5 min	Tempo de bloqueio inicial.
Lockout máximo	<code>T_max</code>	30 min	5–60 min	Teto do bloqueio progressivo.
Janela recorrência	<code>W_rec</code>	15 min	10–30 min	Janela para lockouts consecutivos.
Política crescimento	–	2^k	–	$T_{\text{lockout}} = \min(T_{\text{max}}, T_{\text{base}} \times 2^k)$
Tempo acionamento	<code>T_acio</code>	5 s	3–10 s	Tempo de liberação da tranca.
Debounce	<code>T_deb</code>	50 ms	20–100 ms	Janela de debounce por software.
Limiar bateria	<code>V_low</code>	3,3 V	3,0–3,5 V	Tensão abaixo da qual alerta é emitido.
Janela admin	<code>W_adm</code>	60 s	30–120 s	Tempo após autenticação para permitir troca de senha.

11 Critérios de Aceitação e Testes

Os critérios de aceitação definem os testes que devem ser realizados para validar o correto funcionamento do sistema. Cada teste está vinculado a um ou mais requisitos funcionais.

Tabela 9: Critérios de aceitação e procedimentos de teste.

ID	Teste	RF	Procedimento	Resultado esperado
T-01	Debounce	RF-01	Pressionar cada botão firmemente uma única vez.	Exatamente um evento por pressionamento. Segurar não gera múltiplos dígitos.
T-02	Timeout	RF-02	Digitar 3 dígitos e aguardar $T_{inatividade}$.	Buffer zerado automaticamente, e o sistema retorna a S1.
T-03	Confirmar < 6	RF-15	Digitar 4 dígitos e pressionar Confirmar.	Não valida, não destrava, e fornece feedback de erro leve.
T-04	Bloqueio	RF-03	Inserir $N_{tentativas}$ senhas incorretas.	Lockout por $T_{lockout}$, com entradas ignoradas.
T-05	Bloqueio prog.	RF-04	Provocar dois lockouts dentro de W_{rec} .	Segundo lockout dura $T_{base} \times 2$.
T-06	Anti-marcas	RF-07	Tentar inserir senha sem completar desafio.	Sistema não permite entrada da senha.
T-07	Stealth	RF-14	Ativar stealth e executar operações com som.	Nenhum beep em nenhum evento.
T-08	Bateria baixa	RF-11	Simular tensão abaixo de V_{low} .	Alerta sonoro/visual emitido.
T-09	Persistência	RF-16	Alterar a senha, remover a bateria e reconectar.	Senha e flags preservados.
T-11	Tranc. autom.	RF-17	Autenticar, abrir porta e fechá-la.	Sensor aciona tranca automaticamente.
T-12	Autenticação	RF-05	Fluxo completo: despertar $\rightarrow$ desafio $\rightarrow$ senha $\rightarrow$ confirmar.	Tranca liberada, com feedback de sucesso.

12 Considerações Finais

O Projeto Talos apresenta uma solução completa para um cofre eletrônico baseado em ESP32, abordando desde a especificação de requisitos até os critérios de validação. A arquitetura em duas camadas (Fluxo de Dados e Unidade de Controle) proporciona modularidade e facilita tanto a implementação quanto a manutenção do firmware.

Os mecanismos de segurança implementados, como bloqueio progressivo, desafio anti-marcas, comparação em tempo constante e armazenamento por hash, representam um conjunto robusto de defesas contra as principais ameaças a sistemas de controle de acesso.

A gestão de energia, com uso extensivo do modo deep sleep do ESP32, viabiliza a operação prolongada por bateria, atendendo à meta de 2 anos de autonomia. A persistência de dados com esquema A/B e CRC32 garante a integridade das informações críticas mesmo diante de falhas de energia durante a gravação.

Para replicação do projeto, recomenda-se seguir a tabela de conexões do Capítulo 8, implementar o firmware seguindo a máquina de estados do Capítulo 6, e validar o sistema por meio dos testes do Capítulo 11. O framework ESP-IDF da Espressif fornece todas as APIs necessárias (`gpio`, `timer`, `nvs`, `adc`, `esp_random`) para a implementação completa.

Referências

ESPRESSIF SYSTEMS. *ESP32 Technical Reference Manual*. Disponível em: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf. Acesso em: 2025.

ESPRESSIF SYSTEMS. *ESP-IDF Programming Guide*. Disponível em: <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/>. Acesso em: 2025.

ESPRESSIF SYSTEMS. *ESP32 Datasheet*. Disponível em: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf. Acesso em: 2025.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. *NBR 6023: Informação e documentação – Referências – Elaboração*. Rio de Janeiro: ABNT, 2018.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. *NBR 14724: Informação e documentação – Trabalhos acadêmicos – Apresentação*. Rio de Janeiro: ABNT, 2011.

GANSSE, J. *A Guide to Debouncing*. 2008. Disponível em: <https://www.ganssle.com/debouncing.htm>. Acesso em: 2025.